Record - Week 04

Topic: MongoDB and Mongoose

Name: _____ Roll No: _____ Date: _____

Aim

To perform CRUD operations on a MongoDB database using Mongoose models and controllers.

Software Requirements

MongoDB Community Server, MongoDB Compass, Node.js, Mongoose

Procedure

1. Write dbConfig.js to connect with mongoose.connect.
2. Define the User schema in models/User.js.
3. Implement create, read, update and delete in userController.js.
4. Verify stored data in MongoDB Compass.

Program

dbConfig.js

```
const mongoose = require("mongoose");

const MONGO_URI = process.env.MONGO_URI || "mongodb://127.0.0.1:27017/fs_lab";

async function connectDB() {
  try {
    await mongoose.connect(MONGO_URI);
    console.log("MongoDB connected");
  } catch (err) {
    console.error("MongoDB connection failed:", err.message);
    process.exit(1);
  }
}

module.exports = connectDB;
```

models/User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    age: { type: Number, min: 16, max: 100 },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

controllers/userController.js

```
const User = require("../models/User");

// CREATE
exports.createUser = async (req, res) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

// READ
exports.getUsers = async (req, res) => {
  const users = await User.find().sort({ createdAt: -1 });
  res.json(users);
};

exports.getUser = async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({ error: "User not found" });
  res.json(user);
};

// UPDATE
exports.updateUser = async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, {
    new: true,
    runValidators: true,
  });
  if (!user) return res.status(404).json({ error: "User not found" });
  res.json(user);
};

// DELETE
exports.deleteUser = async (req, res) => {
  const user = await User.findByIdAndDelete(req.params.id);
  if (!user) return res.status(404).json({ error: "User not found" });
```

```
  res.json({ message: "User deleted" });
};
```

Output

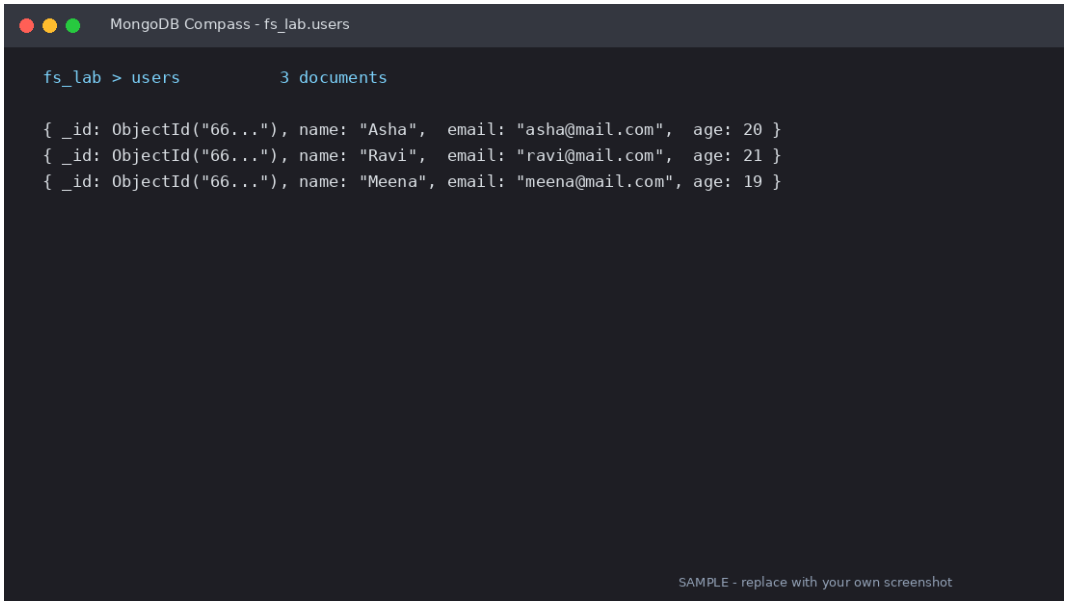

Fig: mongodb compass db

Fig: crud operations output

Result

The program was executed successfully and the expected output was obtained.